



UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE CIENCIAS DE LA INGENIERÍA Y TECNOLOGÍA – FCITEC
UNIDAD VALLE DE LAS PALMAS

Ingeniería en Software y Tecnologías Emergentes

Patrones de Software

Meta 3.4 Desarrollar un software para los patrones de diseño: Singleton, comando, adaptador y fachada que sean utilizados en casos prácticos.

REPORTE DE PRUEBAS UNITARIAS

Patrones de Diseño: Singleton, comando, adaptador y fachada

Millan Nuñez Jiselle

Rodriguez Dominguez Gerardo Adrian

Santos Diaz Christopher Alexander

5to. Semestre

554

Mtra. Abigail Moreno

20 de Marzo del 2026

Este reporte que hicimos documenta la implementación y validación de cuatro patrones de diseño clásicos mediante pruebas unitarias automatizadas en Python. Los patrones implementados son los mencionados en la práctica, Singleton, garantiza una única instancia global de una clase. Command, encapsula acciones como objetos para desacoplar emisor y receptor. Adapter, permite que interfaces incompatibles trabajen juntas. Por último, Facade, proporciona una interfaz simplificada a un subsistema complejo.

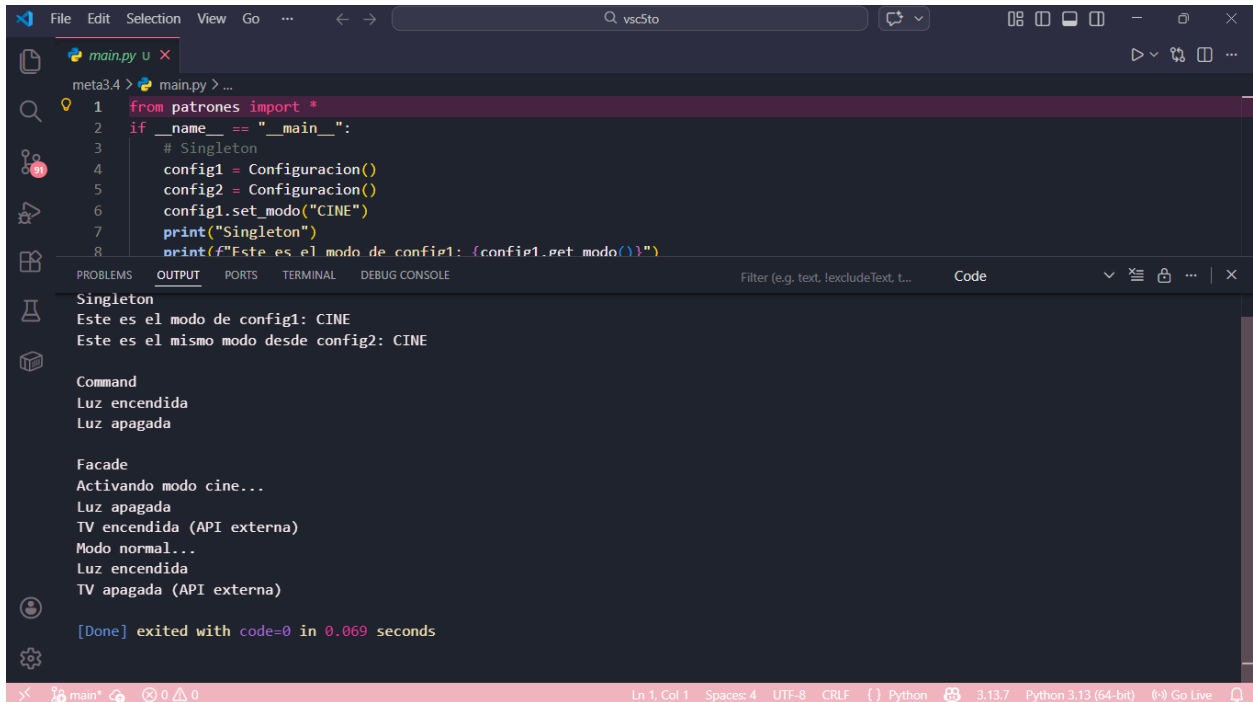
El caso práctico realizado consiste en un sistema de hogar inteligente donde el usuario puede activar modos predefinidos en este caso el modo cine y normal, esto a través de un control remoto, gestionando luces y televisor con APIs heterogéneas.

El proyecto se compone de los siguientes archivos que serán proporcionados con este reporte.

- `patrones.py` — define todas las clases que implementan los cuatro patrones.
- `main.py` — ejecuta el caso práctico demostrando cada patrón en secuencia.
- `test.py` — contiene las pruebas unitarias con el módulo `unittest` de Python.

Ejecución del main

La siguiente imagen muestra la salida completa al ejecutar el main.py, evidenciando el comportamiento correcto de los cuatro patrones:



```
meta3.4 > main.py > ...
1 from patrones import *
2 if __name__ == "__main__":
3     # Singleton
4     config1 = Configuracion()
5     config2 = Configuracion()
6     config1.set_mod0("CINE")
7     print("Singleton")
8     print(f"Este es el modo de config1: {config1.get_mod0()}")

PROBLEMS OUTPUT PORTS TERMINAL DEBUG CONSOLE Filter (e.g. text, lexcludeText, t... Code
Singleton
Este es el modo de config1: CINE
Este es el mismo modo desde config2: CINE

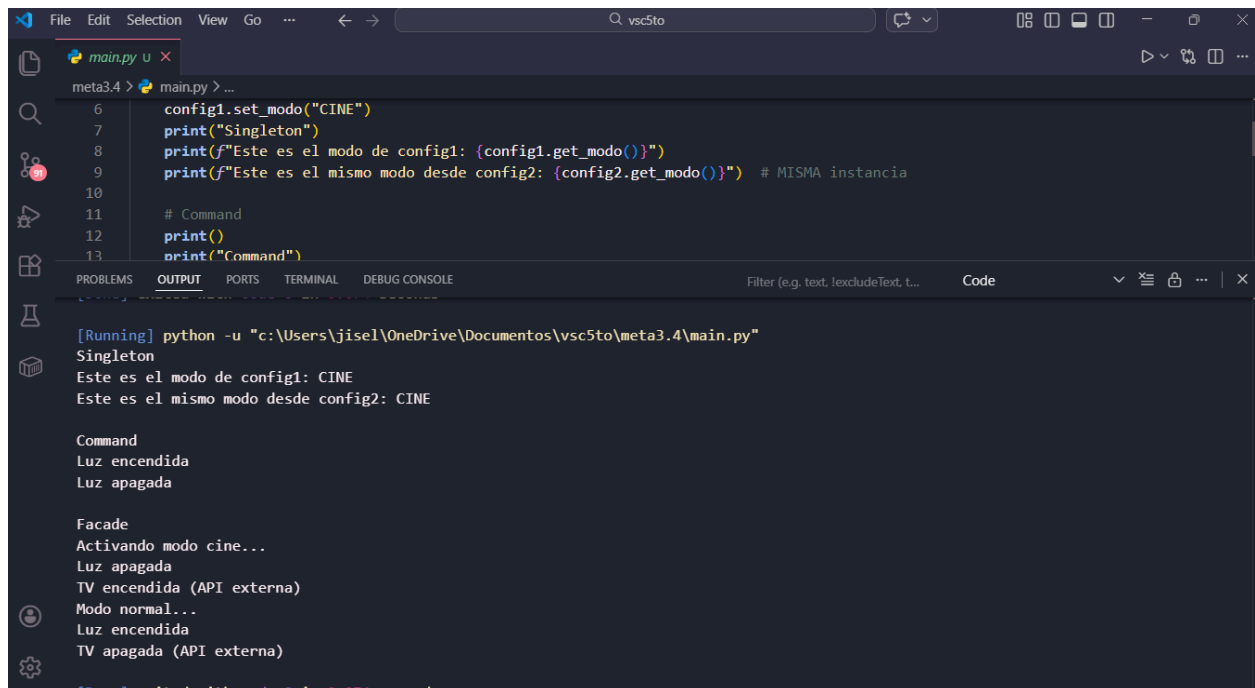
Command
Luz encendida
Luz apagada

Facade
Activando modo cine...
Luz apagada
TV encendida (API externa)
Modo normal...
Luz encendida
TV apagada (API externa)

[Done] exited with code=0 in 0.069 seconds
```

1. Patrón Singleton

Este patrón, restringe la instanciación de una clase a un único objeto, la implementación sobrescribe el método especial `__new__` para devolver siempre la misma instancia almacenada en el atributo de clase `_instancia`, la clase configuración almacena el modo de operación del hogar como normal, cine, etc. cualquier cambio realizado desde una referencia es inmediatamente visible desde cualquier otra, evitando estados inconsistentes.



```
File Edit Selection View Go ... vsc5to
main.py U x
meta3.4 > main.py > ...
6 config1.set_modo("CINE")
7 print("Singleton")
8 print(f"Este es el modo de config1: {config1.get_modo()}")
9 print(f"Este es el mismo modo desde config2: {config2.get_modo()}") # MISMA instancia
10
11 # Command
12 print()
13 print("Command")

PROBLEMS OUTPUT PORTS TERMINAL DEBUG CONSOLE
Filter (e.g. text, lexcludeText, t...) Code

[Running] python -u "c:\Users\jisel\OneDrive\Documentos\vsc5to\meta3.4\main.py"
Singleton
Este es el modo de config1: CINE
Este es el mismo modo desde config2: CINE

Command
Luz encendida
Luz apagada

Facade
Activando modo cine...
Luz apagada
TV encendida (API externa)
Modo normal...
Luz encendida
TV apagada (API externa)

[Done] exited with code 0 in 0.034 seconds
```

En la prueba de instancia única, se verificó que `config1` y `config2` apunten al mismo objeto en memoria, y en la prueba de estado compartido, al cambiar el modo a cine en una instancia, la otra refleja el cambio automáticamente, el resultado fue exitoso.

test_instancia_unica	✓	Verifica que config1 is config2 sea True
test_comparticion_estado	✓	Cambia modo desde config1 y lo lee desde config2

2. Patrón Comando

Este patron encapsula las acciones de encendido y apagado de luces como objetos, permitiendo que un control remoto las ejecute de forma secuencial, se validó que el comando encender luz invoque correctamente el método del objeto luz, en la prueba de cola de comandos, el control remoto almacenó y ejecutó múltiples comandos en el orden correcto, el resultado fue exitoso.



The screenshot shows a Visual Studio Code editor window with a file named `main.py` open. The code in the editor is as follows:

```
11 # Command
12 print()
13 print("Command")
14 luz = Luz()
15 encender = EncenderLuz(luz)
16 apagar = ApagarLuz(luz)
17
18 control = ControlRemoto()
```

Below the editor, the Output window is visible, showing the execution of the script. The output is:

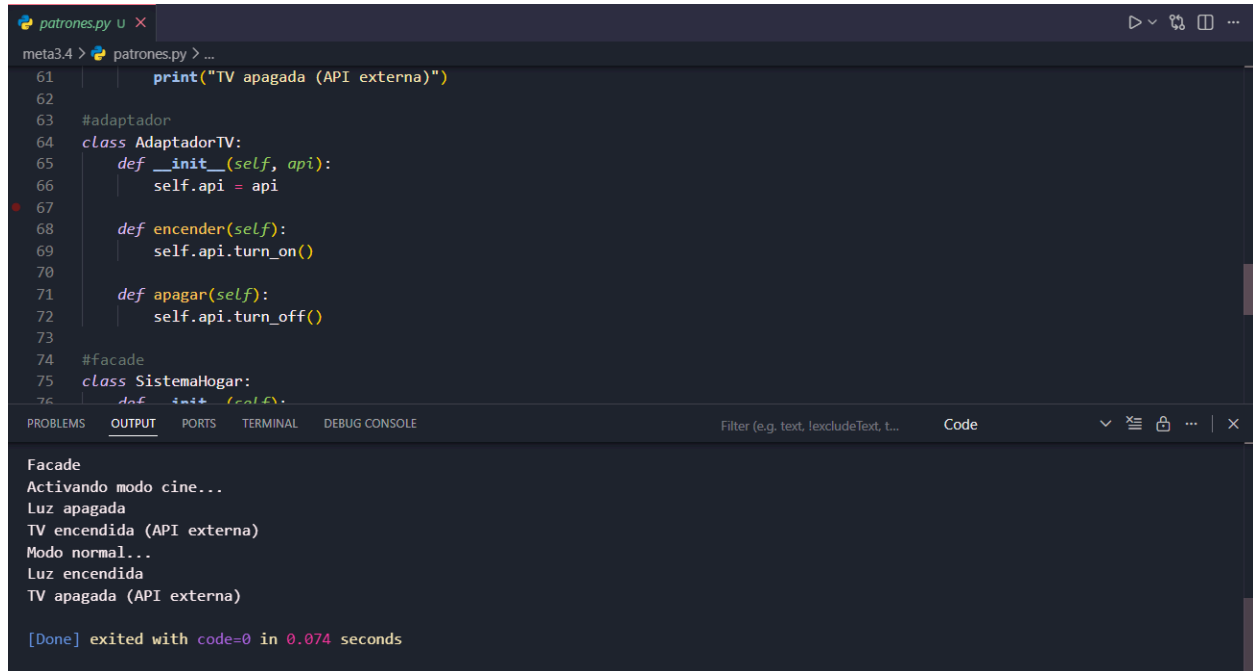
```
[Running] python -u "c:\Users\jisel\OneDrive\Documentos\vsc5to\meta3.4\main.py"
Singleton
Este es el modo de config1: CINE
Este es el mismo modo desde config2: CINE

Command
Luz encendida
Luz apagada
```

test_ejecucion_comando	✓	Ejecuta EncenderLuz y verifica salida 'Luz encendida'
test_control_remoto	✓	Agrega ambos comandos al control y verifica ambas salidas

3. Patrón Adaptador

Este patrón permite que la clase API externa TV sea compatible con nuestra interfaz de hogar, actuando como un traductor entre el cliente y la clase adaptada, en la prueba de adaptación, el adaptador TV llamó exitosamente a los métodos de la API externa sin modificar el código original de la misma.



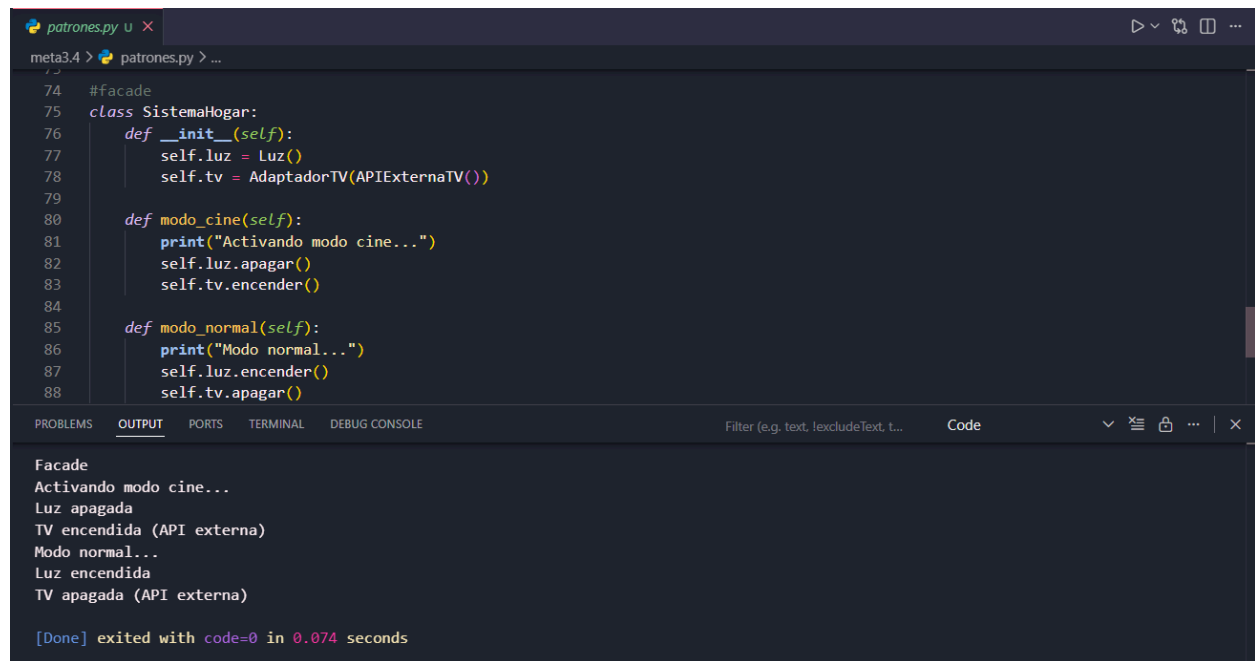
```
patrones.py x
meta3.4 > patrones.py > ...
61     print("TV apagada (API externa)")
62
63     #adaptador
64     class AdaptadorTV:
65     def __init__(self, api):
66         self.api = api
67
68     def encender(self):
69         self.api.turn_on()
70
71     def apagar(self):
72         self.api.turn_off()
73
74     #facade
75     class SistemaHogar:
76     def __init__(self):
77
Facade
Activando modo cine...
Luz apagada
TV encendida (API externa)
Modo normal...
Luz encendida
TV apagada (API externa)
[Done] exited with code=0 in 0.074 seconds
```

test_adaptador_tv	✓	Llama encender/apagar en AdaptadorTV y verifica mensajes de APIExternaTV
-------------------	---	--

La prueba comprueba que al invocar los métodos del adaptador se activa correctamente la API externa, validando tanto TV encendida como TV apagada en la salida capturada.

4. Patrón Fachada

Este patrón proporciona una interfaz simplificada, sistema hogar, para controlar varios dispositivos, luz y TV, simultáneamente. En la prueba modo cine, se confirmó que un solo llamado apaga las luces y enciende la TV. En la prueba modo normal, se confirmó que el sistema restaura el estado de los dispositivos. El usuario no necesita saber que hay una luz, un adaptador y una API separada, simplemente activa el modo deseado a través de la fachada.



```
74 #facade
75 class SistemaHogar:
76     def __init__(self):
77         self.luz = Luz()
78         self.tv = AdaptadorTV(APIExternaTV())
79
80     def modo_cine(self):
81         print("Activando modo cine...")
82         self.luz.apagar()
83         self.tv.encender()
84
85     def modo_normal(self):
86         print("Modo normal...")
87         self.luz.encender()
88         self.tv.apagar()
```

Facade
Activando modo cine...
Luz apagada
TV encendida (API externa)
Modo normal...
Luz encendida
TV apagada (API externa)

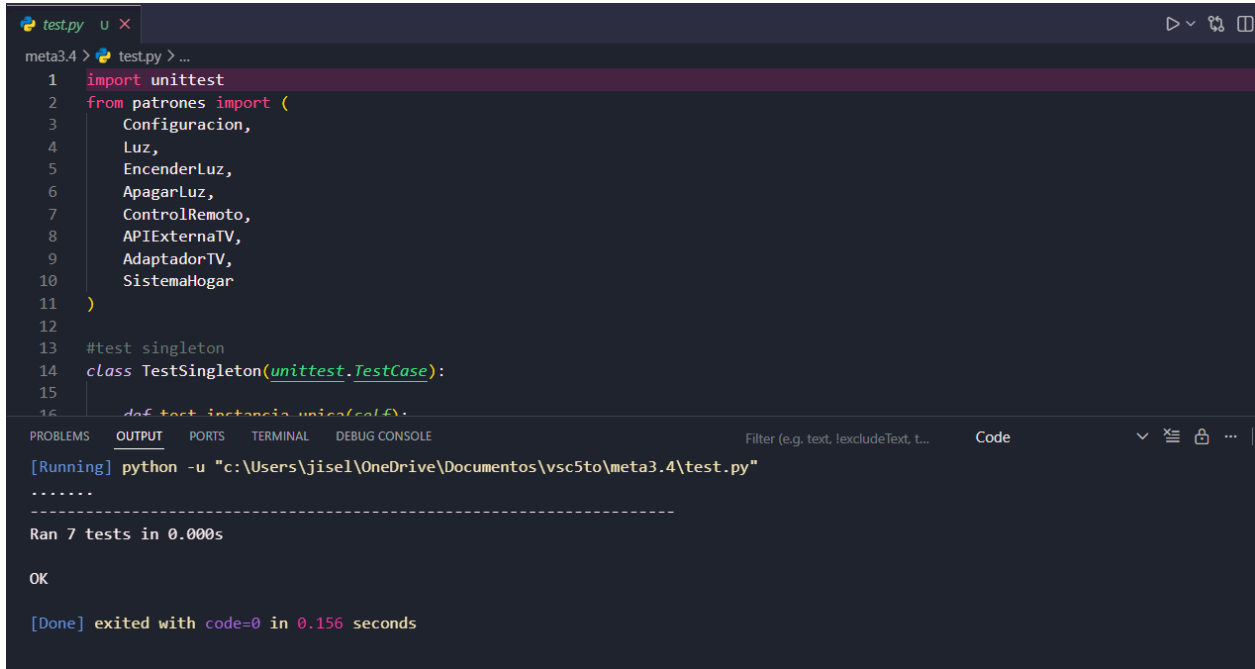
[Done] exited with code=0 in 0.074 seconds

test_modos_cine	✓	Verifica 'Activando modo cine', 'Luz apagada' y 'TV encendida'
test_modos_normal	✓	Verifica 'Modo normal', 'Luz encendida' y 'TV apagada'

Cada prueba de la fachada valida tres aserciones simultáneas, confirmando que la fachada coordina correctamente todos los subsistemas. Esto prueba la integración end-to-end de los patrones Adapter y Facade en un flujo real.

Pruebas generales archivo test.py

La siguiente imagen muestra la ejecución completa de la suite de pruebas unitarias con el framework unittest de Python:



The image shows a Visual Studio Code editor window with a file named `test.py` open. The code in the editor is as follows:

```
1 import unittest
2 from patrones import (
3     Configuracion,
4     Luz,
5     EncenderLuz,
6     ApagarLuz,
7     ControlRemoto,
8     APIExternaTV,
9     AdaptadorTV,
10    SistemaHogar
11 )
12
13 #test singleton
14 class TestSingleton(unittest.TestCase):
15
16     def test_instancia_unica(self):
```

Below the code editor, the `OUTPUT` panel is visible, showing the execution of the tests. The output is as follows:

```
[Running] python -u "c:\Users\jisel\OneDrive\Documentos\vsc5to\meta3.4\test.py"
.....
-----
Ran 7 tests in 0.000s

OK

[Done] exited with code=0 in 0.156 seconds
```

En resumen los 7 casos de prueba ejecutados fueron exitosos.

Conclusiones

En conclusión, la implementación de los cuatro patrones de diseño fue exitosa, logramos que los 7 casos de prueba pasaran sin errores en una sola ejecución. El patrón singleton, mediante `__new__`, simplificó el control global, mientras que el patrón comando desacopló al emisor del receptor para facilitar la escalabilidad del sistema, el patrón adaptador integró una API externa de forma transparente y la fachada redujo la complejidad operativa a solo un par de líneas de código. Esta metodología no solo ayuda a optimizar el rendimiento técnico del software, sino que también establece una base sólida para futuras expansiones, permitiendo que el sistema crezca de manera modular y organizada.